

Air University



Cryptography (Lab)

Year 2026

Mr. Omer Ali

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: March 26, 2026

Lab # 08**1. Code**

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
#include <cctype>
using namespace std;

// 5x5 Playfair Key Matrix
// Maps keyword + alphabet (I/J merged) to grid
char matrix[5][5];
// Row/col positions for each letter (I/J merged)
int rowOf[26], colOf[26];

void buildMatrix(const string &keyword)
{
    // Initialize used array, merge J into I
    bool used[26] = {};
    used['J' - 'A'] = true;

    string seq = "";
    // Build seq from unique keyword letters
    for (char c : keyword)
    {
        c = toupper(c);
        if (!isalpha(c))
            continue;
        if (c == 'J')
            c = 'I';
        if (!used[c - 'A'])
        {
            used[c - 'A'] = true;
            seq += c;
        }
    }
    // Append remaining letters A-Z
    for (char c = 'A'; c <= 'Z'; c++)
    {
        if (!used[c - 'A'])
            seq += c;
    }

    // Fill matrix row-major and update positions
    for (int i = 0; i < 25; i++)
    {
        int r = i / 5, col = i % 5;
        matrix[r][col] = seq[i];
        rowOf[seq[i] - 'A'] = r;
    }
}
```

```
        colOf[seq[i] - 'A'] = col;
    }
}

void printMatrix()
{
    cout << "\n Playfair Key Matrix:\n";
    cout << " +---+---+---+---+---+---+\n";
    // Print each row with borders
    for (int r = 0; r < 5; r++)
    {
        cout << " | ";
        // Print each column
        for (int c = 0; c < 5; c++)
            cout << matrix[r][c] << " | ";
        cout << "\n +---+---+---+---+---+---+\n";
    }
}

// Prepare plaintext into digraphs: clean, uppercase, I/J merged, pad doubles/single with X
string prepareText(const string &text)
{
    string clean = "";
    // Clean to uppercase alpha only, merge J=I
    for (char c : text)
    {
        if (!isalpha(c))
            continue;
        c = toupper(c);
        if (c == 'J')
            c = 'I';
        clean += c;
    }

    string result = "";
    int i = 0;
    // Form digraphs, pad with X if double or odd length
    while (i < (int)clean.size())
    {
        char a = clean[i];
        if (i + 1 == (int)clean.size())
        {
            // Odd length: pad with X
            result += a;
            result += 'X';
            i++;
        }
        else
        {
            char b = clean[i + 1];
            if (a == b)
```

```
        {
            // Same letters: insert X
            result += a;
            result += 'X';
            i++;
        }
        else
        {
            // Valid digraph
            result += a;
            result += b;
            i += 2;
        }
    }
}
return result;
}

// Prepare ciphertext for decryption: uppercase alpha only, I/J merged, no padding removal
string prepareCiphertext(const string &text)
{
    string result = "";
    // Clean to uppercase alpha only, merge J=I
    for (char c : text)
    {
        if (!isalpha(c))
            continue;
        c = toupper(c);
        if (c == 'J')
            c = 'I';
        result += c;
    }
    return result;
}

// Process single digraph: row shift, col shift, or rectangle swap based on mode (+1
// encrypt, -1 decrypt)
pair<char, char> processDigraph(char a, char b, int mode)
{
    int rA = rowOf[a - 'A'], cA = colOf[a - 'A'];
    int rB = rowOf[b - 'A'], cB = colOf[b - 'A'];

    if (rA == rB)
    {
        // Same row: shift columns
        cA = (cA + mode + 5) % 5;
        cB = (cB + mode + 5) % 5;
    }
    else if (cA == cB)
    {
        // Same column: shift rows
        rA = (rA + mode + 5) % 5;
    }
}
```

```
        rB = (rB + mode + 5) % 5;
    }
    else
    {
        // Rectangle: swap columns
        swap(cA, cB);
    }
    return {matrix[rA][cA], matrix[rB][cB]};
}

string processText(const string &text, int mode)
{
    string result = "";
    // Process each digraph pair
    for (int i = 0; i < (int)text.size(); i += 2)
    {
        auto [a, b] = processDigraph(text[i], text[i + 1], mode);
        result += a;
        result += b;
    }
    return result;
}

// Remove padding X's from decrypted text (end/trailing doubles)
string removePadding(const string &text)
{
    string result = "";
    // Remove terminal X or X between same letters
    for (int i = 0; i < (int)text.size(); i++)
    {
        if (i == (int)text.size() - 1 && text[i] == 'X')
            break; // Skip final pad X
        if (text[i] == 'X' && i > 0 && i < (int)text.size() - 1 && text[i - 1] == text[i +
1])
            continue; // Skip pad X between doubles
        result += text[i];
    }
    return result;
}

// Menu handler functions
void handleEncryption()
{
    string keyword, plaintext;

    cout << "\n--- Encryption ---\n";
    cout << "Enter keyword    : ";
    cin.ignore();
    getline(cin, keyword);

    if (keyword.empty())
    {
```

```
        cout << "Error: Keyword cannot be empty!\n";
        return;
    }

    cout << "Enter plaintext : ";
    getline(cin, plaintext);

    if (plaintext.empty())
    {
        cout << "Error: Plaintext cannot be empty!\n";
        return;
    }

    buildMatrix(keyword);
    printMatrix();

    string prepared = prepareText(plaintext);
    string encrypted = processText(prepared, +1);

    cout << "\nResult:\n";
    cout << "Keyword          : " << keyword << "\n";
    cout << "Original Text       : " << plaintext << "\n";
    cout << "Prepared Digraphs: ";
    // Print digraphs
    for (int i = 0; i < (int)prepared.size(); i += 2)
        cout << prepared.substr(i, 2) << " ";
    cout << "\nCiphertext          : " << encrypted << "\n";
}

void handleDecryption()
{
    string keyword, ciphertext;

    cout << "\n--- Decryption ---\n";
    cout << "Enter keyword   : ";
    cin.ignore();
    getline(cin, keyword);

    if (keyword.empty())
    {
        cout << "Error: Keyword cannot be empty!\n";
        return;
    }

    cout << "Enter ciphertext: ";
    getline(cin, ciphertext);

    if (ciphertext.empty())
    {
        cout << "Error: Ciphertext cannot be empty!\n";
        return;
    }
}
```

```
// Validate even length for digraphs
string cleaned = prepareCiphertext(ciphertext);
if (cleaned.size() % 2 != 0)
{
    cout << "Error: Ciphertext length must be even (Playfair encrypts in pairs)!\n";
    return;
}

buildMatrix(keyword);
printMatrix();

string decrypted = removePadding(processText(cleaned, -1));

cout << "\nResult:\n";
cout << "Keyword          : " << keyword << "\n";
cout << "Ciphertext          : " << ciphertext << "\n";
cout << "Decrypted Text      : " << decrypted << "\n";
}

// Main program: menu loop for encrypt/decrypt/exit
int main()
{
    int choice;

    // Loop until exit
    do
    {
        cout << "\n===== Playfair Cipher Program =====\n";
        cout << "1. Encrypt\n";
        cout << "2. Decrypt\n";
        cout << "3. Exit\n";
        cout << "Enter your choice (1-3): ";
        cin >> choice;

        if (choice == 1)
        {
            handleEncryption();
        }
        else if (choice == 2)
        {
            handleDecryption();
        }
        else if (choice == 3)
        {
            cout << "\nThank you for using Playfair Cipher!\n";
        }
        else
        {
            cout << "Invalid choice! Please enter 1, 2, or 3.\n";
        }
    }
```

```
} while (choice != 3);  
  
return 0;  
}
```

Code Explanation:

- I. **Building the Key Matrix (buildMatrix):** The program creates a 5×5 grid using a keyword. It adds unique letters from the keyword first and merges J into I. After that, it fills in the rest of the letters in the alphabet. Two lookup arrays, rowOf[] and colOf[], make it easy to find any letter's row and column when encrypting or decrypting.
- II. **Printing the Matrix (printMatrix):** Once the program gets the keyword, it prints the 5×5 grid with ASCII borders. This helps users check that the keyword is correct before any encryption or decryption takes place.
- III. **Preparing Plaintext (prepareText):** The program processes the plaintext by converting it to uppercase and removing non-alphabetic characters. It replaces J with I. Then, it pairs the letters. If there are two identical letters, it inserts an X between them (e.g., LL becomes LX L). If the number of letters is odd after pairing, it adds an X at the end to make it even.
- IV. **Preparing Ciphertext (prepareCiphertext):** For decryption, the program simply removes non-alphabetic characters and replaces J with I. It does not add any padding since the ciphertext already has valid pairs.
- V. **Processing Digraphs (processDigraph):** The program applies the main Playfair rules for three cases: when both letters are in the same row, they shift right for encryption or left for decryption; when they are in the same column, they shift down or up; when they form a rectangle, each letter moves to the row of the other letter.
- VI. **Processing Text (processText):** The prepared string is processed in pairs of letters, sending each pair to processDigraph() along with a mode flag. The results come together to create either the complete ciphertext or the decrypted text.
- VII. **Removing Padding (removePadding):** After decryption, any X characters that were added are removed, including an X at the end and any X between identical letters (e.g., L-X-L) to restore the original plaintext.
- VIII. **Encryption Handler (handleEncryption):** The program asks the user for a keyword and plaintext. It checks for empty inputs. It creates the matrix, prepares digraphs, performs encryption, and shows information like the keyword, original text, prepared digraphs, and final ciphertext.
- IX. **Decryption Handler (handleDecryption):** The program requests a keyword and ciphertext from the user, ensures the input is valid, and checks that the cleaned ciphertext has an even length. An odd length suggests possible errors in the input. If everything is validated, the ciphertext is decrypted, the padding is removed, and the result is displayed.
- X. **Main Menu Loop (main):** A do-while loop presents three options: Encrypt, Decrypt, Exit. The program directs users to the correct handler and captures invalid inputs, showing error messages as needed. The menu structure is like that of the Vigenère cipher.

2. Output

```
===== Playfair Cipher Program =====
1. Encrypt
2. Decrypt
3. Exit
Enter your choice (1-3): 1

--- Encryption ---
Enter keyword   : playfairexample
Enter plaintext : hidethegoldinthetreestump

Playfair Key Matrix:
+---+---+---+---+---+
| P | L | A | Y | F |
+---+---+---+---+---+
| I | R | E | X | M |
+---+---+---+---+---+
| B | C | D | G | H |
+---+---+---+---+---+
| K | N | O | Q | S |
+---+---+---+---+---+
| T | U | V | W | Z |
+---+---+---+---+---+

Result:
Keyword       : playfairexample
Original Text  : hidethegoldinthetreestump
Prepared Digraphs: HI DE TH EG OL DI NT HE TR EX ES TU MP
Ciphertext    : BMODZBXDNABEKUDMUIXMMOUIVF
```

```

===== Playfair Cipher Program =====
1. Encrypt
2. Decrypt
3. Exit
Enter your choice (1-3): 2

--- Decryption ---
Enter keyword   : playfairexample
Enter ciphertext: BMODZBXDNABEKUDMUIXMMOUIVF

Playfair Key Matrix:
+---+---+---+---+---+
| P | L | A | Y | F |
+---+---+---+---+---+
| I | R | E | X | M |
+---+---+---+---+---+
| B | C | D | G | H |
+---+---+---+---+---+
| K | N | O | Q | S |
+---+---+---+---+---+
| T | U | V | W | Z |
+---+---+---+---+---+

Result:
Keyword       : playfairexample
Ciphertext    : BMODZBXDNABEKUDMUIXMMOUIVF
Decrypted Text : HIDETHEGOLDINTHETREESTUMP

===== Playfair Cipher Program =====
1. Encrypt
2. Decrypt
3. Exit
Enter your choice (1-3): 3

Thank you for using Playfair Cipher!

```

3. Observation

The program successfully builds a 5×5 matrix from any keyword, handling duplicate letters and merging I and J correctly. For example, encrypting "HIDETHEGOLDINTHETREESTUMP" with the keyword "PLAYFAIREXAMPLE" produces the digraph sequence "HI DE TH EG OL DI NT HE TR EX ES TU MP" showing the insertion of X between repeated letters. Using the same keyword and ciphertext for decryption yields "HIDETHEGOLDINTHETREESTUMP," with the padding Xs correctly removed. The matrix displays after each keyword entry, allowing users to confirm the cipher's state before proceeding.

4. Results

Encryption effectively changes plaintext into Playfair ciphertext by following the three digraph rules: shifting for the same row, shifting for the same column, and swapping corners in a rectangle. Decryption reverses this process, restoring the original message after padding removal. Both processes produce successful results, with the decrypted message matching the original plaintext. The check for even-length ciphertext helps catch errors caused by incorrect input. The menu loop handles all options, including invalid choices.

5. Advantages

- **Digraph Encryption:** Using pairs of letters makes frequency analysis harder compared to simpler ciphers like Caesar or Atbash.
- **No Repeated Ciphertext for Repeated Plaintext:** Inserting an X between identical letter pairs stops the same plaintext pair from mapping back to itself, addressing vulnerabilities in basic substitution ciphers.
- **Single Shared Key:** The same 5×5 matrix is used for both encryption and decryption, allowing secure sharing of just the keyword.
- **Compact Implementation:** The three Playfair rules cover all digraph situations without needing extra handling aside from merging I and J, making the algorithm simple to implement and review.

6. Disadvantages

- **Vulnerable to Digraph Frequency Analysis:** While looking at single letters won't work, analyzing pairs of letters (digraphs) can break the code. In English, common pairs like TH, HE, and IN appear often. An attacker with enough coded text can study these pairs to potentially figure out the key.
- **Indistinguishability of I and J:** The cipher combines I and J into one letter, which can confuse the decoded text. For example, both INJECT and INIECT will look the same when coded, making it unclear what the intended letters are.
- **Limited Character Support:** The method does not include numbers, spaces, or punctuation when preparing the message. This limitation can disrupt the original message structure, making it hard for the recipient to see where words begin and end.
- **Fixed 5×5 Matrix Size:** The cipher uses a 25-letter alphabet, which limits its flexibility. It cannot include larger character sets like numbers, special symbols, or Unicode without changing the matrix and the related digraph rules.

7. Compare with Vigenère Cipher

Aspect	Playfair Cipher	Vigenère Cipher
Encryption Unit	Works with pairs of letters (digraphs)	Works with single letters
Key Structure	Uses a 5×5 matrix built from a keyword	Uses a repeating keyword for a shift sequence
Frequency Analysis	Resists single-letter analysis but is weak to digraph analysis	Susceptible to single-letter analysis
I/J Handling	Combines I and J in one cell, causing confusion	Treats all 26 letters distinctly
Non-Alpha Characters	Ignores them completely in preparation	Keeps them unchanged in the output
Implementation Complexity	More complex due to matrix construction and handling the digraph rules	Simpler, as it uses a straightforward formula

Both ciphers are symmetric encryption methods with historical significance and may not meet today's security standards. The Playfair Cipher offers better resistance to basic frequency analysis by using letter pairs, while the Vigenère Cipher is easier to use and retains all original characters, including spaces and punctuation.

8. Conclusion

The Playfair Cipher uses pairs of letters for symmetric encryption through a 5×5 matrix created from a keyword. It has three substitution rules: row shift, column shift, and rectangle swap. This approach improves upon simple substitution ciphers by using letter pairs, making basic frequency analysis less effective. However, it is vulnerable to digraph frequency analysis and does not differentiate between I and J. Therefore, its main

use is for educational purposes. The cipher serves as an important example of how polyalphabetic and positional substitution techniques led to modern block cipher designs.

- End of Lab Task -